

Lab3 Report

By Pranav Upadhyay

1. Lab Objectives

- Simulate a differential drive robot (TurtleBot3 Waffle Pi) in Gazebo
- Implement and tune two PID controllers — one for angular velocity, one for linear velocity
- Move the robot from its current pose to a reference pose (x_r , y_r , θ_r)
- Support two control modes — sequential (mode 0) and simultaneous (mode 1)

2. Code Snippet and explanation

pid_controller.py

```
#!/usr/bin/env python3

import rospy
import math
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
from std_msgs.msg import Float32MultiArray
from tf.transformations import euler_from_quaternion

# — PID Gains —
KP_ANG, KI_ANG, KD_ANG = 2.0, 0.001, 0.3
KP_LIN, KI_LIN, KD_LIN = 0.8, 0.001, 0.1

MAX_LINEAR    = 0.22    # m/s
MAX_ANGULAR   = 2.84    # rad/s
DIST_THRESH   = 0.05    # metres
ANGLE_THRESH  = 0.05    # radians

# — Shared state —
robot = {'x': 0.0, 'y': 0.0, 'yaw': 0.0}
goal  = {'xr': None, 'yr': None, 'thetar': None, 'mode': None, 'active': False}
phase = {'current': 'idle'}

ang_pid = {'integral': 0.0, 'prev_error': 0.0, 'prev_time': None,
           'kp': KP_ANG, 'ki': KI_ANG, 'kd': KD_ANG, 'max': MAX_ANGULAR}
lin_pid = {'integral': 0.0, 'prev_error': 0.0, 'prev_time': None,
           'kp': KP_LIN, 'ki': KI_LIN, 'kd': KD_LIN, 'max': MAX_LINEAR}

cmd_pub = None

# — Helpers —
```

```

def normalize_angle(a):
    while a > math.pi: a -= 2 * math.pi
    while a < -math.pi: a += 2 * math.pi
    return a

def pid_compute(pid, error, now):
    dt = 0.05 if pid['prev_time'] is None else max((now -
pid['prev_time']).to_sec(), 1e-4)
    pid['integral'] += error * dt
    output = (pid['kp'] * error
              + pid['ki'] * pid['integral']
              + pid['kd'] * (error - pid['prev_error']) / dt)
    pid['prev_error'] = error
    pid['prev_time'] = now
    return max(-pid['max'], min(pid['max'], output))

def pid_reset(pid):
    pid['integral'] = 0.0
    pid['prev_error'] = 0.0
    pid['prev_time'] = None

def stop():
    cmd_pub.publish(Twist())

def dist_to_goal():
    return math.hypot(goal['xr'] - robot['x'], goal['yr'] - robot['y'])

def bearing_to_goal():
    return math.atan2(goal['yr'] - robot['y'], goal['xr'] - robot['x'])

# — Callbacks —

def odom_cb(msg):
    robot['x'] = msg.pose.pose.position.x
    robot['y'] = msg.pose.pose.position.y
    q = msg.pose.pose.orientation
    _, _, robot['yaw'] = euler_from_quaternion([q.x, q.y, q.z, q.w])

def goal_cb(msg):
    if len(msg.data) < 4:
        rospy.logwarn("Reference pose needs 4 values: xr yr thetar mode")
        return
    goal['xr'], goal['yr'], goal['thetar'] = msg.data[0], msg.data[1],
msg.data[2]
    goal['mode'] = int(round(msg.data[3]))

```

```

goal['active'] = True
pid_reset(ang_pid)
pid_reset(lin_pid)
phase['current'] = 'rotate_to_goal' if goal['mode'] == 0 else 'drive'
rospy.loginfo(f"New goal → x={goal['xr']:.2f} y={goal['yr']:.2f} "
              f"θ={goal['thetar']:.2f} mode={goal['mode']}")

# — Control loops —

def run_mode0(now):
    dist = dist_to_goal()
    angle_err = normalize_angle(bearing_to_goal() - robot['yaw'])
    final_err = normalize_angle(goal['thetar'] - robot['yaw'])
    cmd = Twist()

    if phase['current'] == 'rotate_to_goal':
        if abs(angle_err) < ANGLE_THRESH:
            rospy.loginfo("Facing goal. Starting drive.")
            pid_reset(lin_pid)
            phase['current'] = 'drive'
        else:
            cmd.angular.z = pid_compute(ang_pid, angle_err, now)

    elif phase['current'] == 'drive':
        if dist < DIST_THRESH:
            rospy.loginfo("Position reached. Rotating to final angle.")
            pid_reset(ang_pid)
            phase['current'] = 'rotate_final'
        else:
            cmd.angular.z = pid_compute(ang_pid, angle_err, now)
            cmd.linear.x = pid_compute(lin_pid, dist, now)

    elif phase['current'] == 'rotate_final':
        if abs(final_err) < ANGLE_THRESH:
            rospy.loginfo("Goal reached (mode 0).")
            stop()
            goal['active'] = False
            phase['current'] = 'idle'
        else:
            cmd.angular.z = pid_compute(ang_pid, final_err, now)

    cmd_pub.publish(cmd)

def run_mode1(now):
    dist = dist_to_goal()

```

```

final_err = normalize_angle(goal['thetar'] - robot['yaw'])
cmd = Twist()

if phase['current'] == 'drive':
    if dist < DIST_THRESH:
        rospy.loginfo("Position reached. Correcting final angle.")
        pid_reset(ang_pid)
        phase['current'] = 'rotate_final'
    else:
        angle_err = normalize_angle(bearing_to_goal() - robot['yaw'])
        cmd.angular.z = pid_compute(ang_pid, angle_err, now)
        cmd.linear.x = pid_compute(lin_pid, dist, now)

elif phase['current'] == 'rotate_final':
    if abs(final_err) < ANGLE_THRESH:
        rospy.loginfo("Goal reached (mode 1).")
        stop()
        goal['active'] = False
        phase['current'] = 'idle'
    else:
        cmd.angular.z = pid_compute(ang_pid, final_err, now)

cmd_pub.publish(cmd)

# — Main —

def main():
    global cmd_pub
    rospy.init_node('pid_controller', anonymous=False)
    cmd_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=10)
    rospy.Subscriber('/odom', Odometry, odom_cb)
    rospy.Subscriber('/reference_pose', Float32MultiArray, goal_cb)

    rospy.loginfo("PID Controller node started.")
    rate = rospy.Rate(20)
    while not rospy.is_shutdown():
        if goal['active']:
            now = rospy.Time.now()
            run_mode0(now) if goal['mode'] == 0 else run_mode1(now)
        rate.sleep()

if __name__ == '__main__':
    try:
        main()
    except rospy.ROSInterruptException:

```

pass

- Subscribes to `/reference_pose` for the target (`xr`, `yr`, `θr`, `mode`) and `/odom` for current pose; publishes Twist commands to `/cmd_vel` at 20 Hz
- `odom_cb` - extracts `x`, `y` from `msg.pose.pose.position` and converts orientation quaternion to yaw via `euler_from_quaternion`; updates robot dict every odometry message
- `goal_cb` - unpacks `msg.data` into goal dict, sets `goal['active'] = True`, calls `pid_reset` on both PIDs to clear stale history, sets `phase['current']` to `rotate_to_goal` (mode 0) or `drive` (mode 1)
- `pid_compute` - computes `dt` from timestamp difference, accumulates error into integral, computes derivative as $(\text{error} - \text{prev_error}) / dt$, returns clamped output $K_p * e + K_i * \int e + K_d * (de/dt)$
- `pid_reset` - zeroes integral, `prev_error`, and `prev_time` in the given PID dict; prevents leftover values from a previous move corrupting the next one
- `normalize_angle` - subtracts or adds 2π until angle is within $[-\pi, \pi]$; ensures robot always turns the shortest direction rather than spinning the long way around
- `dist_to_goal` - returns $\sqrt{(x_r - x)^2 + (y_r - y)^2}$; used as the error input to the linear PID and as the phase transition condition
- `bearing_to_goal` - returns $\text{atan2}(y_r - y, x_r - x)$; the absolute world-frame angle pointing from robot to goal; subtracted from yaw to get `angle_err`
- `run_mode0` - phase `rotate_to_goal`: feeds `angle_err` to angular PID until $< \text{ANGLE_THRESH}$; phase `drive`: runs both PIDs, angular keeps robot pointed at goal while linear closes the distance; phase `rotate_final`: feeds `thetar - yaw` to angular PID until final heading is reached
- `run_mode1` - phase `drive`: runs angular and linear PIDs simultaneously, robot curves toward goal; phase `rotate_final`: same as mode 0, corrects heading after position is reached
- `stop` - publishes an empty Twist (all zeros) to `/cmd_vel`; called when goal is fully reached to halt the robot
- `main` - initializes node, sets up publisher and subscribers, runs 20 Hz loop calling `run_mode0` or `run_mode1` whenever `goal['active']` is True

`motion_planner.py`

```
#!/usr/bin/env python3

import rospy
import math
from std_msgs.msg import Float32MultiArray
from nav_msgs.msg import Odometry
from tf.transformations import euler_from_quaternion

DIST_THRESH = 0.1    # metres
ANGLE_THRESH = 0.1    # radians

robot = {'x': 0.0, 'y': 0.0, 'yaw': 0.0}

pose_pub = None

# — Helpers —
```

```

def normalize_angle(a):
    while a > math.pi: a -= 2 * math.pi
    while a < -math.pi: a += 2 * math.pi
    return a

def goal_reached(xr, yr, thetar):
    ep = math.hypot(xr - robot['x'], yr - robot['y'])
    et = abs(normalize_angle(thetar - robot['yaw']))
    return ep <= DIST_THRESH and et <= ANGLE_THRESH

# — Callback —————

def odom_cb(msg):
    robot['x'] = msg.pose.pose.position.x
    robot['y'] = msg.pose.pose.position.y
    q = msg.pose.pose.orientation
    _, _, robot['yaw'] = euler_from_quaternion([q.x, q.y, q.z, q.w])

# — User input & publish —————

def get_user_input():
    print("\n--- Enter reference pose ---")
    try:
        xr = float(input(" x_r (metres) : "))
        yr = float(input(" y_r (metres) : "))
        thetar = float(input(" θ_r (radians) : "))
        mode = int(input(" mode (0 or 1) : "))
        if mode not in (0, 1):
            print(" /\! Mode must be 0 or 1.")
            return None
        return xr, yr, thetar, mode
    except ValueError:
        print(" /\! Invalid input – numbers only.")
        return None

def publish_goal(xr, yr, thetar, mode):
    msg = Float32MultiArray()
    msg.data = [float(xr), float(yr), float(thetar), float(mode)]
    pose_pub.publish(msg)
    rospy.loginfo(f"Published goal → x={xr:.2f} y={yr:.2f} θ={thetar:.2f} mode={mode}")

# — Main —————

```

```

def main():
    global pose_pub
    rospy.init_node('motion_planner', anonymous=False)
    pose_pub = rospy.Publisher('/reference_pose', Float32MultiArray,
queue_size=10)
    rospy.Subscriber('/odom', Odometry, odom_cb)
    rospy.sleep(1.0)
    rospy.loginfo("Motion Planner node started.")

    rate = rospy.Rate(10)
    while not rospy.is_shutdown():
        result = get_user_input()
        if result is None:
            continue

        xr, yr, thetar, mode = result
        publish_goal(xr, yr, thetar, mode)
        print(f"\n Driving to ({xr:.2f}, {yr:.2f}, {thetar:.2f}) [mode={mode}]
...")

        while not rospy.is_shutdown():
            if goal_reached(xr, yr, thetar):
                ep = math.hypot(xr - robot['x'], yr - robot['y'])
                et = abs(normalize_angle(thetar - robot['yaw']))
                print(f"\n Goal reached! ep={ep:.4f} m eθ={et:.4f} rad")
                break
            rate.sleep()

if __name__ == '__main__':
    try:
        main()
    except rospy.ROSInterruptException:
        pass

```

- Subscribes to /odom to track current robot pose; publishes Float32MultiArray of [xr, yr, θr, mode] to /reference_pose
- odom_cb - same as in pid_controller, keeps local robot dict updated so goal_reached always has a fresh pose to check against
- get_user_input - prompts user for xr, yr, thetar, mode via input(), casts to float/int, validates mode is 0 or 1, returns None on bad input so main re-prompts without crashing
- publish_goal - packs [xr, yr, thetar, mode] into Float32MultiArray.data and publishes to /reference_pose; this is what triggers goal_cb in pid_controller
- goal_reached - computes $ep = \sqrt{(xr-x)^2 + (yr-y)^2}$ and $et = |\text{normalize_angle}(\text{thetar} - \text{yaw})|$; returns True only when both are within 0.1, matching the grader's exact bounds

- `normalize_angle` — same wrap logic as in `pid_controller`; needed here so heading error doesn't incorrectly read as large when angles are near $\pm\pi$
- `main` — loops: call `get_user_input` → `publish_goal` → enter inner loop polling `goal_reached` at 10 Hz → once reached, print final `ep` and `eθ` values → return to outer loop for next goal

3. Screenshots



